We will first test out our self-attention model on MNIST, but we ultimately plan to use it as our deep Q-network in game playing, so the only difference between a DQN and an image classifier is that the dimensionality of the inputs and outputs will be different—everything in between can remain the same.

10.3.1. Transformed MNIST

Before we build the model itself, we need to prepare the data and create some functions to preprocess the data so that it is in the right form for our model. For one, the raw MNIST images are grayscale pixel arrays with values from 0 to 255, so we need to normalize those values to be between 0 and 1, or the gradients during training will be too variable and training will be unstable. Because MNIST is so easy, we can also strain our model a bit more by adding noise and perturbing the images randomly (e.g., random translations and rotations). This will also allow us to assess translational and rotational invariance. These preprocessing functions are defined in the following listing.

Listing 10.1. Preprocessing functions

```
import numpy as np
from matplotlib import pyplot as plt
import torch
from torch import nn
import torchvision as TV

mnist_data = TV.datasets.MNIST("MNIST/", train=True,
transform=None,\
                                        target_transform=None,
download=True)1
mnist_test = TV.datasets.MNIST("MNIST/", train=False,
transform=None,\
                                        target_transform=None,
download=True)2


def add_spots(x,m=20,std=5,val=1):
3
    mask = torch.zeros(x.shape)
    N = int(m + std * np.abs(np.random.randn()))
    ids = np.random.randint(np.prod(x.shape),size=N)
    mask.view(-1)[ids] = val
```